

Data ingestion job and GraphQL with Limit Search

- - Overview
 - Main Functions
 - Environment Setup and Dependencies
 - Configuration Management
 - Secret Management
 - Data Extraction from EDP
 - Timestamp Comparison and Incremental Updates
 - Elasticsearch Document Construction
 - Data Indexing and Storage
 - Utility Functions
 - Data Processing Flow
 - Error Handling and Resilience
 - Network and Security Configuration
 - Performance Optimization
 - Monitoring and Troubleshooting
- GraphQL API and Limit Search
 - Overview
 - Query syntax
 - Limit search
 - Limit search related intents
 - Limit search conditions syntax

Overview

This document explains the main logic and functions used for ingesting company specific data in to the elastic search database. The customer.ai chatbot backend uses the elastic company data to search and narrow down the results to specific set of companies relevant to the question. This AWS Glue job handles the ingestion of customer data from the Enterprise Data Platform (EDP) GraphQL API into Elasticsearch for the PC377 AI Smart Search application. The job processes data columns that could be leveraged for searching, generates embeddings using machine learning models, and maintains an up-to-date search index with periodic data synchronization.

The job operates in a proxied NextEra Energy environment and includes comprehensive error handling, timestamp-based incremental updates, and automatic dependency management for required Python packages.

Gluejob name : pc377-smart-search-ingestion-edp-job

Main Functions

Environment Setup and Dependencies

The main execution block configures proxy settings for NextEra Energy's network environment and dynamically installs required packages including Elasticsearch 8.12.1 and requests library. This ensures the job can access external APIs through the corporate proxy while maintaining the necessary dependencies for execution.

Configuration Management

The `load_config()` function reads configuration data from the `edp_ingestion.yaml` file, providing centralized management of job parameters and settings. This function includes error handling to gracefully exit if configuration files are unavailable or corrupted.

Secret Management

The `get_secret()` function retrieves API credentials and connection details from AWS Secrets Manager. It accesses the secret "pc377-ai-smart-search-api-secrets" containing EDP API keys, Elasticsearch URL, and authentication credentials required for secure data access across the ingestion pipeline.

Data Extraction from EDP

The `get_edp_data()` function serves as the primary data extraction engine, querying the Enterprise Data Platform's GraphQL API for customer information. It implements intelligent pagination to handle large datasets, processing both insert and update timestamps to ensure comprehensive data coverage while respecting API rate limits.

This function retrieves extensive customer data including regional short-term values for various ISO markets (CAISO, ERCOT, ISO-NE, MISO, NYISO, PJM, SPP, SERC, WECC), customer identification numbers, preferred names, CDP organizational information, industry classifications, credit ratings, account executive details, and geographical information.

Timestamp Comparison and Incremental Updates

The `get_elastic_compare_data()` function implements intelligent data synchronization by comparing timestamps between incoming EDP data and existing Elasticsearch documents. This prevents unnecessary updates and ensures only modified or new records are processed, significantly improving job efficiency and reducing processing time. The function handles non-existent documents gracefully, treating them as new records, and includes comprehensive error handling to maintain job stability during comparison operations.

Elasticsearch Document Construction

The `build_elastic_data()` function transforms raw EDP data into structured Elasticsearch documents. It maps all customer data fields, generates text embeddings using the sentence-transformers multi-qa-distilbert-cos-v1 model, and prepares documents for indexing with proper field validation and default value handling.

The embedding generation combines customer descriptions and preferred names to create semantic search capabilities, enabling intelligent query matching beyond exact text searches.

Data Indexing and Storage

The `elastic_ingestion()` function handles the actual insertion of processed documents into the Elasticsearch index "smart-search-pc377-zeus". This function provides the final step in the data pipeline, ensuring transformed and enriched customer data is available for search operations.

Utility Functions

The codebase includes several utility functions for operational support: `list_current_directory()` for debugging file system access, `uninstall()` for package management, and `generate_hash()` for creating unique identifiers when needed.

Data Processing Flow

The job follows a systematic data processing approach starting with environment configuration and dependency installation. It retrieves authentication secrets, establishes connections to both EDP GraphQL API and Elasticsearch, then processes data in paginated batches to handle large customer datasets efficiently.

For each company record, the job performs timestamp comparison to determine if updates are necessary, transforms the data into the required Elasticsearch structure, generates semantic embeddings for enhanced search capabilities, and indexes the final document into the search infrastructure.

Error Handling and Resilience

The job implements comprehensive error handling throughout the data pipeline including graceful exits for configuration failures, retry logic for API communications, fallback mechanisms for embedding generation failures, and detailed logging for troubleshooting and monitoring purposes.

The timestamp-based incremental update mechanism includes a smart termination feature that stops processing when 500 consecutive records with identical or older timestamps are encountered, preventing unnecessary processing of unchanged data.

Network and Security Configuration

The job operates within NextEra Energy's secure network environment using corporate proxy settings for all external communications. All sensitive credentials are managed through AWS Secrets Manager, ensuring secure access to both EDP APIs and Elasticsearch infrastructure without exposing authentication details in the code.

Performance Optimization

The job includes several performance optimizations including pagination-based data retrieval to manage memory usage, timestamp-based incremental updates to minimize processing overhead, batch processing capabilities for efficient Elasticsearch operations, and intelligent termination logic to prevent unnecessary iterations through unchanged data.

The embedding generation is performed selectively, combining relevant text fields to create meaningful semantic representations while handling cases where source data may be incomplete or unavailable.

Columns ingested

- `caiso_shrt_trm_val`
- `ercot_shrt_trm_val`
- `iso_ne_shrt_trm_val`
- `miso_shrt_trm_val`
- `nyiso_shrt_trm_val`
- `pjm_shrt_trm_val`
- `spp_shrt_trm_val`
- `serc_shrt_trm_val`
- `wecc_shrt_trm_val`
- `ces_cust_nbr`
- `prfrd_nm`
- `cdp_org_nm`
- `cdp_actvs_txt`
- `cdp_sectrs_txt`
- `cdp_prmry_actvy_nm`
- `cdp_prmry_sectr_nm`

- iva_indstry_prvs_yr_nm
- gics_sub_ind_prvs_yr_nm
- neer_vrtcl_prvs_yr_nm
- prfrd_indstry_prvs_yr_nm
- c_and_i_acct_exec_orig_leadtr_nm
- c_and_i_acct_exec_nm
- crdt_ratg_s_and_p_val
- hq_city_state_prvs_yr_nm
- msci_tgt_shrt_desc
- etl_insrtdtmm
- etl_updt_dttm

Monitoring and Troubleshooting

Comprehensive logging throughout the job execution provides visibility into processing progress, error conditions, and performance metrics. The job logs include page numbers during pagination, document processing confirmations, timestamp comparisons for debugging, embedding generation status, and detailed error messages for troubleshooting.

For operational support, the job provides clear indicators of processing completion, handles various failure scenarios gracefully, and maintains detailed execution logs accessible through AWS CloudWatch for monitoring and analysis.

GraphQL API and Limit Search

Overview

We are documenting this EDP API to ensure our development team has comprehensive visibility into the search infrastructure that powers customer insights. This documentation serves as the foundation for connecting to EDP API, and the GraphQL query.

This API is a query tool used by the Customer.AI chatbot backend, to get all data relevant data from Redshift DB.

Connection

- **API URL:**
 - Dev: <https://mvt3t4mhqbdkjm5bftx42dvxq.appsync-api.us-east-1.amazonaws.com/graphql>
- **Authentication type:** api-key
- **Headers:**
 - x-api-key: API key
 - Content-Type: application/json

Query syntax

```
query MyQuery {
  get_ces_customer(
    eff_end_dttm: { eq: "9999-12-31 00:00:00.000" }, # remove the company duplicates
    "" + condition + "" { # all needed conditions separate by comma

    pagination {
      page
      limit
    }
    data {
      "" + fields + "" #returning field's names
    }
  }
}
```

Limit search

Limit search related intents

- `min_value` (float)
The numeric value that the user's question indicates is the minimum value
- `max_value` (float)
The numeric value that the user's question indicates is the maximum value
- `sort_direction` (string)
The ascending or descending sorting direction indicated by the user

Limit search conditions syntax

- Sorting: `sort: ["field name.sort_direction"]`
 - top: sort direction desc
 - bottom: sort direction asc
- Less than: field name: `{lte: "max_value"}`
- Greater than: field name: `{gte: "min_value"}`